Golang Hands-on Assignment: Product Struct with Metrics

Objective:
Implement a Go program that defines a Product struct with fields id, name, and price. Create an array of products in main. Write functions to accept user input for products, display all products, and calculate key metrics such as average price.

Assignment Instructions

1. **Define a Product struct** with fields:
    - id (integer)
    - name (string)
    - price (float64)
2. **Write a function to accept product details from the user** and return a slice of Products.
    - The function should prompt the user for the number of products to enter.
    - For each product, ask for input for id, name, and price.
3. **Write a function to display the products** in a readable format.
4. **Write a function to compute metrics**:
    - Average price of all products.

Sample Expected Output

Enter number of products: 3

Product 1:
Enter ID: 101
Enter Name: Apple
Enter Price: 60.5

Product 2:
Enter ID: 102
Enter Name: Banana
Enter Price: 30

Product 3:
Enter ID: 103
Enter Name: Carrot
Enter Price: 25.75

Product List:
ID: 101, Name: Apple, Price: 60.50
ID: 102, Name: Banana, Price: 30.00
ID: 103, Name: Carrot, Price: 25.75

Average Price: 38.08

Golang Hands-on Assignment: Understanding Interfaces and Polymorphism

Objective:
Implement a Go program to illustrate interfaces and polymorphism. Define an Employee interface with a method to calculate salary. Create two kinds of employees—Permanent and Contract—with distinct salary formulas. Use polymorphism to print and process different employee types uniformly.

Assignment Instructions

1. Define the Interface and Structs

- **Interface:**
  - Employee: Declares a single method CalculateSalary() that returns the employee's salary as float64.
- **Structs:**
  - PermanentEmployee
    - Fields: id (int), name (string), basic (float64), hra (float64), da (float64), ta (float64)
    - hra = 15% of basic
    - da = 20% of basic
    - ta = 5% of basic
  - ContractEmployee
    - Fields: id (int), name (string), hours (int), perHourRate (float64)

2. Implement Methods

- Each struct (PermanentEmployee, ContractEmployee) must implement the CalculateSalary() method:
  - **PermanentEmployee:**
    Salary=basic+hra+da+ta (calculate hra, da and ta bas
  - **ContractEmployee:**
    Salary=hours×perHourRate

3. Polymorphic Processing

- Create a function printEmployee that accepts any Employee and prints its details, including using the CalculateSalary() method. This demonstrates polymorphism: the same function can print different employee types via the shared interface method.

Note:- Accept the basic from permanent employee. Calculate other salary related values based on basic. Id and name values must be accepted for each struct type.